

DISEÑO TÉCNICO — MÓDULO DE IA Y PLANIFICACIÓN

Agente Generativo de Planificación Académica

Puntos 3 y 4 del Documento Técnico

Materia: Introducción a la Programación con IA
Ciclo II – 2026 | Ingeniería de Software y Negocios Digitales
Docente: Ing. Christopher Cañas

Stack: FastAPI (Python) + Supabase (PostgreSQL) + Claude API (Anthropic)
Repositorio: github.com/chriss212/academic-planner-api

Fecha: 9 de julio de 2026

3. Integración con IA Generativa (capa separada)

Esta sección documenta la capa de integración con el modelo Claude de Anthropic. Todo el código de esta capa vive exclusivamente en el backend FastAPI — nunca en el frontend — conforme a RNF-02 (no exponer API keys) y al requisito explícito de separación de capas del enunciado (sección 4.2).

3.1 Estructura del módulo de IA

La capa de IA se organiza en dos carpetas con responsabilidades distintas dentro del backend:

```
app/
├── ia/                                ← integración con el proveedor de IA
│   ├── prompt_builder.py             ← arma el prompt con contexto estructurado
│   ├── ia_client.py                  ← llama al SDK de Anthropic
│   ├── response_validator.py          ← valida y parsea el JSON de respuesta
│   └── replan_service.py              ← replanificación con contexto del plan anterior
└── services/                          ← lógica de planificación (ya existe en el repo)
    ├── planning.py                   ← reglas de negocio: sobrecarga, conflictos
    ├── validation.py                 ← validaciones generales de datos
    └── replanning.py                 ← orquesta el flujo de replanificación
```

3.2 Contexto que se envía a la IA — prompt_builder.py

Antes de armar el prompt, se construye un objeto de contexto con toda la información que el modelo necesita para generar un plan válido. Este objeto se lee de la base de datos Supabase usando los modelos y routers ya implementados en el repo (tasks, availability_blocks, constraints).

```
contexto = {
    "usuario": {
        "id": "uuid-del-usuario",
        "preferencias": ["evitar sesiones > 90 min", "prioridad académica sobre personal"]
    },
    "tareas": [
        {
            "id": "t1",
            "titulo": "Ensayo de Historia",
            "categoria": "academica",
            "fecha_limite": "2026-07-15T23:59:00",
            "prioridad": 5,                ← viene de tasks.priority (1-5)
            "esfuerzo_estimado_min": 180, ← effort_hours * 60
            "estado": "pending",          ← viene de tasks.status
            "movible": true
        }
    ]
}
```

```

],
"disponibilidad": [
  {"dia": "2026-07-10", "bloques": [{"inicio": "14:00", "fin": "17:00"}]}
],
"restricciones": [
  {"tipo": "blocked_time", "descripcion": "Trabajo lunes 8:00-13:00"},
  {"tipo": "fixed_task", "tarea_id": "t1"},
  {"tipo": "max_session_hours", "valor_min": 90}
],
"fecha_generacion": "2026-07-09T10:00:00"
}

```

3.2.1 System prompt fijo (versionado)

El system prompt es fijo, versionado en el catálogo de prompts, y nunca se genera dinámicamente. Incluye las siguientes reglas que el modelo debe respetar:

- No ignorar ninguna fecha límite declarada.
- No asignar tareas fuera de los bloques de disponibilidad declarados.
- Respetar restricciones de tipo fixed_task y max_session_hours.
- Si no es posible cumplir todas las restricciones, indicarlo en el campo viabilidad.
- Responder únicamente en JSON válido, sin texto adicional, sin markdown.
- No inventar tareas ni datos que no fueron proporcionados.

3.3 Llamada al modelo, parsing y validación — ia_client.py + response_validator.py

El flujo de llamada al modelo sigue estos pasos en orden fijo:

1. prompt_builder.py → construye contexto desde BD
2. ia_client.py → llama a anthropic SDK con system + user message
3. response_validator.py
 - Nivel 1: ¿es JSON parseable? ¿están los campos obligatorios?
 - Nivel 2: ¿los bloques asignados están en disponibilidad declarada?
4. Si pasa → plan marcado como 'propuesto' → revisión humana
5. Si falla → manejo de errores (sección 4.3)

3.3.1 Llamada al SDK de Anthropic — ia_client.py

```

import anthropic
from app.core.config import settings

client = anthropic.Anthropic(api_key=settings.anthropic_api_key)

async def generate_plan(system_prompt: str, user_context: dict) -> str:
    response = client.messages.create(

```

```

model=settings.ai_model,    # lee de .env → "claude-sonnet-4-6"
max_tokens=2048,
system=system_prompt,
messages=[{"role": "user", "content": str(user_context)}]
)
return response.content[0].text

```

3.3.2 Esquema de respuesta esperada (contrato fijo)

La IA debe devolver siempre este JSON. El esquema se define como modelo Pydantic en `response_validator.py` y se usa para validar cada respuesta antes de guardarla:

```

{
  "version_plan": "1",
  "viabilidad": "viable | viable_con_ajustes | no_viable",
  "plan": [
    {
      "tarea_id": "t1",
      "dia": "2026-07-10",
      "bloque_inicio": "14:00",
      "bloque_fin": "15:30",
      "orden": 1
    }
  ],
  "justificacion": "Texto explicando el orden y prioridades.",
  "riesgos": [{"tarea_id": "t1", "riesgo": "fecha_limite_ajustada", "detalle": "..."}],
  "conflictos": [{"tipo": "sobrecarga", "dia": "2026-07-11", "detalle": "..."}],
  "recomendaciones": ["Mover tarea t3 si es posible."]
}

```

3.3.3 Validación en dos niveles — `response_validator.py`

Nivel	Qué valida	Ejemplo de fallo	RF cubierto
Nivel 1 — Estructural	¿Es JSON parseable? ¿Están version_plan, viabilidad, plan y justificacion? ¿Todo tarea_id del plan existe en las tareas enviadas?	La IA devolvió texto plano en vez de JSON	RNF-04
Nivel 2 — Reglas de negocio	¿Los bloques asignados están dentro de la disponibilidad declarada? ¿Se respetaron las tareas fixed_task? ¿Ninguna sesión excede max_session_hours?	La IA asignó una tarea en horario bloqueado	RF-08, RNF-04

3.4 Lógica de replanificación — replan_service.py (RF-07)

Cuando cambia una tarea, fecha límite, prioridad o disponibilidad, el sistema no genera un plan desde cero. reutiliza prompt_builder agregando un bloque de contexto de cambio:

```
{
  "tipo_evento": "cambio_disponibilidad | tarea_nueva | tarea_modificada |
tarea_atrasada",
  "detalle_cambio": "Usuario marcó t1 como atrasada y agregó 2h el 2026-07-12",
  "plan_anterior_version": "1",
  "plan_anterior": { "...": "JSON del plan previo" }
}
```

Reglas fijas de la replanificación:

- Siempre se envía el plan anterior como referencia.
- Se incrementa version_plan (de '1' a '2', etc.).
- El motivo del cambio se guarda en el historial junto con la nueva versión.
- La respuesta pasa por el mismo flujo de validación de dos niveles — la replanificación NO se salta la validación.

3.5 Catálogo de prompts

El catálogo documenta cada prompt utilizado, su versión, el objetivo, y ejemplos de entrada y salida. Es un entregable obligatorio del proyecto.

ID	Nombre	Versión	Objetivo	Módulo
P-01	System prompt base	v1.0	Define reglas fijas que el modelo debe respetar en toda generación	ia/prompt_builder.py
P-02	User prompt — generación inicial	v1.0	Envía contexto completo (tareas + disponibilidad + restricciones) para generar el plan semanal	ia/prompt_builder.py
P-03	User prompt — replanificación	v1.0	Agrega bloque de cambio al contexto del P-02 para solicitar una nueva versión del plan	ia/replan_service.py
P-04	Prompt de corrección de formato	v1.0	Se envía cuando la respuesta no es JSON válido (ERR-IA-001); pide al modelo corregir solo el formato	ia/ia_client.py

Para cada prompt del catálogo se documenta: versión, ejemplo de entrada (contexto completo), ejemplo de salida (respuesta cruda del modelo), y las mejoras aplicadas entre versiones. Esta evidencia alimenta directamente el entregable 'Inventario de prompts' requerido por la rúbrica.

4. Lógica de Planificación y Manejo de Errores

Esta sección documenta las reglas de negocio que se ejecutan sobre los datos del usuario y sobre las respuestas de la IA, el catálogo de errores posibles y su manejo, y los tres escenarios de prueba obligatorios. Todo este comportamiento vive en la carpeta `services/` del backend, que ya existe en el repo pero está pendiente de implementación.

4.1 Reglas de negocio de planificación — `services/planning.py` (RF-08)

Las reglas de negocio se ejecutan en dos momentos distintos: (1) sobre los datos crudos antes de armar el prompt, para advertir a la IA en el contexto mismo; y (2) sobre la respuesta de la IA, como el nivel 2 de validación.

4.1.1 Detección de sobrecarga

```
para cada día d en el rango del plan:
    tiempo_requerido(d) = suma(effort_hours de tareas asignadas a d)
    tiempo_disponible(d) = suma(duración de bloques en availability_blocks para d)
    si tiempo_requerido(d) > tiempo_disponible(d):
        → conflicto 'sobrecarga', día d, déficit = diferencia en horas
```

4.1.2 Detección de fechas en riesgo

```
para cada tarea t:
    si tiempo_planificado_antes_del_limite(t) < effort_hours(t):
        → riesgo 'fecha_limite_en_riesgo'
    si deadline(t) < hoy AND status(t) != 'completed':
        → actualizar status a 'overdue' (usa PATCH /tasks/{id} ya implementado)
```

4.1.3 Detección de conflictos de restricciones

```
para tareas fixed_task:
    si el plan las reubicó → conflicto 'restriccion_violada'
para cada bloque asignado:
    si no está dentro de availability_blocks declarados → conflicto
    'fuera_de_disponibilidad'
para cada sesión asignada:
    si duración > max_session_hours → conflicto 'excede_duracion_maxima'
```

4.2 Validaciones generales de datos — `services/validation.py` (RNF-04)

Campo	Validación	Dónde se aplica	RF/RNF
Fecha límite (deadline)	Fecha válida, no anterior a la fecha de creación de la tarea	TaskCreate schema + services/validation.py	RNF-04
Prioridad (priority)	Entero entre 1 y 5 (Pydantic Field + CheckConstraint en BD)	schemas/task.py + models/task.py	RNF-04
Esfuerzo (effort_hours)	Entero positivo, alerta si > 8 horas en una sola tarea	schemas/task.py + services/validation.py	RNF-04
Disponibilidad	Bloques no solapados; start_time < end_time	schemas/availability.py (model_validator) + BD	RNF-04
Restricciones	Tipo dentro del enum ConstraintType; metadata presente si aplica	schemas/constraint.py	RNF-04
Respuesta IA	Cumple el esquema de la sección 3.3.2; campos obligatorios presentes	ia/response_validator.py	RNF-04
Referencias IA	Todo tarea_id en el plan existe en las tareas enviadas al modelo	ia/response_validator.py (nivel 1)	RNF-04, RF-08

4.3 Manejo de errores — RF-12, RNF-06

Código	Situación	Comportamiento del sistema	RF
ERR-DATA-001	Datos incompletos (sin disponibilidad o sin tareas)	No se llama a la IA. Se devuelve 422 con mensaje específico indicando qué falta.	RF-12
ERR-IA-001	Respuesta no es JSON parseable	Reintento único con prompt de corrección (P-04). Si falla de nuevo: error visible + registro en ai_traces con response_status='error'.	RF-12
ERR-IA-002	Falta un campo obligatorio en el JSON	Igual que ERR-IA-001.	RF-12
ERR-IA-003	viabilidad fuera del enum permitido	Se descarta la respuesta, se marca inválida en ai_traces, se informa al usuario.	RF-12
ERR-IA-004	La IA violó una restricción declarada (nivel 2)	El plan NO se muestra como final. Se marca 'requiere_revision' con la advertencia específica.	RF-08, RF-12
ERR-PLAN-001	Conflicto no resoluble (más trabajo del que cabe)	Plan marcado no_viable. Se listan tareas que no caben. Se sugiere ajustar fechas o alcance.	RF-08, RF-12
ERR-SYS-001	Fallo de conexión con Anthropic (timeout, 5xx)	Mensaje 'servicio no disponible'. No se pierden datos ya guardados.	RF-12

4.4 Diseño de los 3 escenarios de prueba obligatorios

El enunciado exige literalmente estos tres escenarios (sección 4.1). Cada uno ejercita una parte distinta del sistema evaluable por separado durante la defensa.

Escenario 1 — Semana académica normal

Elemento	Detalle
Datos de entrada	5 tareas con prioridades mixtas (1-5), fechas repartidas en la semana, disponibilidad suficiente (3h diarias de lunes a viernes).
Restricciones	1 tarea fixed_task, max_session_hours = 90 min, bloqueo horario 8:00-13:00 lunes a viernes.
Resultado esperado	viabilidad = 'viable', plan completo sin conflictos, justificación explicando el orden por prioridad y cercanía a fecha límite.
Qué se prueba	Camino feliz completo: prompt_builder → ia_client → response_validator nivel 1 y 2 → plan propuesto → aprobación del usuario.

Escenario 2 — Semana con sobrecarga

Elemento	Detalle
Datos de entrada	8-10 tareas, esfuerzo total muy superior a la disponibilidad (se necesitan 25h, solo hay 12h disponibles en la semana).
Restricciones	Mismas del escenario 1 más 2 tareas adicionales urgent con deadline en 2 días.
Resultado esperado	viabilidad = 'viable_con_ajustes' o 'no_viable', conflictos listados por día con déficit calculado, recomendaciones para ajustar alcance.
Qué se prueba	Detección de sobrecarga (sección 4.1.1), manejo de ERR-PLAN-001, y que el sistema indica claramente que el plan no es posible sin eliminar los datos del usuario.

Escenario 3 — Semana con cambio inesperado

Elemento	Detalle
Punto de partida	Plan aprobado del escenario 1 (version_plan = '1', status = 'approved').
Cambio introducido	Se marca la tarea de mayor prioridad como 'overdue' usando PATCH /tasks/{id}, y se elimina un bloque de disponibilidad del miércoles.
Resultado esperado	Se dispara la replanificación (replan_service), version_plan pasa a '2', la justificación referencia el cambio explícitamente, la nueva traza se guarda en ai_traces.
Qué se prueba	Flujo completo de replanificación (RF-07), trazabilidad de versiones (RF-09, RF-10), y que el endpoint PATCH /tasks/{id} ya implementado desencadena correctamente la replanificación.

Para cada escenario se documenta: datos de entrada exactos, prompt final enviado a la IA, respuesta cruda recibida, resultado de la validación, y la decisión del usuario (aprobar/rechazar). Esta evidencia alimenta el entregable 'Evidencia de pruebas' y el 'Catálogo de prompts' requeridos por la rúbrica.

4.5 Persistencia de trazabilidad — ai_traces (RF-09, RF-10, RNF-05)

Cada llamada al modelo genera un registro en la tabla ai_traces (pendiente de implementar en models/ai_trace.py):

Columna	Tipo	Descripción
id	UUID	Identificador del registro
user_id	UUID	Referencia al usuario (para RLS de Supabase)
plan_id	UUID	Referencia al plan generado
version	INT	Versión del plan (1, 2, 3...)
prompt_enviado	JSONB	Contexto completo + system prompt enviados al modelo
respuesta_ia	JSONB	Respuesta cruda del modelo sin procesar
response_status	VARCHAR	valid invalid error (del catálogo ERR-*)
modelo_usado	VARCHAR	Valor de settings.ai_model al momento de la llamada
created_at	TIMESTAMPTZ	Fecha y hora de la generación